

Entregável — Exercício 3.1

Projeto: NovaTech Assistant API | Harness Engineering e Structured Outputs

1. O Desafio e a Solução

Para garantir a fiabilidade determinística das respostas geradas pelo pipeline RAG, implementou-se um **Harness** usando o Zod para validação estruturada (Structured Output). Esta camada atua como um funil de segurança antes do retorno ao utilizador corporativo, bloqueando "alucinações" formativas ou respostas que violam a Política de Devolução (POL-001).

2. Código Original Gerado (GitHub Copilot)

Na primeira iteração de geração com o GitHub Copilot, a IA criou uma estrutura funcional, porém ingénua perante os riscos de um ambiente de IA Generativa. O código gerado foi o seguinte:

src/services/response-validator.ts (V1 - Inicial)

```
import { z } from 'zod';

export const StructuredOutputSchema = z.object({
  answer: z.string(),
  source_document: z.string(),
  confidence_score: z.number(),
});

const NEGATION_TERMS = [
  'não', 'nao', 'proibido', 'proibida', 'negado', 'negada',
  'impossível', 'impossivel', 'não é elegível', 'nao é elegivel',
  'não é elegivel', 'nao é elegível', 'não pode', 'nao pode', 'nunca',
];

const createSafeResponse = () => ({
  answer: 'Desculpe, não posso processar essa solicitação no momento ou a resposta violou nossas políticas de segurança.',
  source_document: 'N/A',
  confidence_score: 0,
});

const containsNegation = (text: string) =>
  NEGATION_TERMS.some((term) => text.toLowerCase().includes(term));

const shouldBlockDangerousReturn = (answer: string) => {
  const normalized = answer.toLowerCase();
  const mentionsDangerousCargo = normalized.includes('carga perigosa');
  const mentionsReturn = normalized.includes('devolução');

  return (
    mentionsDangerousCargo &&
    mentionsReturn &&
    !containsNegation(normalized)
  );
};

const logValidationError = (reason: string, details?: unknown) => {
  console.error('[response-validator] validation failed:', reason, details ?? 'no details');
};
```

```
export const validateAssistantResponse = (rawLlmResponse: unknown) => {
  try {
    const parsed = StructuredOutputSchema.parse(rawLlmResponse);

    if (!parsed.source_document.trim()) {
      logValidationError('source_document is required and cannot be empty', parsed);
      return createSafeResponse();
    }

    if (shouldBlockDangerousReturn(parsed.answer)) {
      logValidationError('blocked answer mentioning carga perigosa and devolução without negation',
        parsed.answer);
      return createSafeResponse();
    }

    return parsed;
  } catch (error) {
    logValidationError('structured output validation failed', error);
    return createSafeResponse();
  }
};
```

3. Code Review: Análise Crítica do Código de IA

Durante a revisão de *Staff Engineer* realizada com o auxílio do Claude, três falhas críticas foram identificadas neste código inicial:

Problema 1: Schema Zod Permissivo

A Falha: O `z.object()` inicial não continha `.strict()`.

O Risco: LLMs frequentemente inventam chaves no JSON (ex: `"bypass_guardrails": true`). O modo padrão do Zod ignora esses extras, permitindo que payloads contaminados por *Prompt Injection* avancem pelas camadas do sistema de forma invisível.

Problema 2: Validação Frágil de Strings

A Falha: O campo `source_document` aceitava strings vazias e delegava a verificação para um `if()` avulso no final da função.

O Risco: O LLM poderia responder com espaços vazios (" ") disfarçando a ausência da citação de fonte. Colocar regras de negócio fora do contrato Zod enfraquece a tipagem e abre margem para regressões futuras caso o `if` seja apagado acidentalmente.

Problema 3: Guardrail Ingênuo (Hardcoded)

A Falha: O uso de `.includes('carga perigosa')` é cego para as riquezas morfológicas da língua portuguesa.

O Risco: Se o LLM escrevesse "cargas perigosas" ou "devoluções", a função de segurança seria imediatamente evadida, autorizando incorretamente um processo proibido por regras de negócio.

4. Código Blindado (Production-Ready)

Abaixo está a versão final do arquivo `response-validator.ts` refatorado, aplicando tipagem estrita e Expressões Regulares robustas, mitigando os problemas encontrados no Code Review.

`src/services/response-validator.ts (V2 - Final)`

```
import { z } from 'zod';

// — Schema —————
// .strict() rejeita qualquer campo extra que o LLM alucinar.
// .min(1).trim() move a guarda de source_document para dentro do contrato de tipo.
export const StructuredOutputSchema = z
  .object({
    answer: z.string().min(1, 'answer não pode ser vazio'),
    source_document: z
      .string()
      .trim()
      .min(1, 'source_document é obrigatório e não pode ser vazio ou whitespace'),
    confidence_score: z.number().min(0).max(1),
  })
  .strict(); // rejeita campos extras – superfície de ataque de prompt injection

export type StructuredOutput = z.infer<typeof StructuredOutputSchema>;

// — Guardrails —————
const NEGATION_REGEX = /\b(n[ãõ]o|proibid[ao]|negad[ao]|imposs[íi]vel|n[ãõ]o\s+[eé]\s+eleg[íi]vel|n[ãõ]o\s+pode|nunca)\b/i;
const DANGEROUS_CARGO_REGEX = /\b(cargas?\s+perigosas?|materiais?\s+perigosos?|produtos?\s+perigosos?|subst[â]ncias?\s+perigosas?)\b/i;
const RETURN_REGEX = /\b(devolu[cç][ãõ]o|devolu[cç][oõ]es|devolv[eia]|devolvid[ao]|devolver)\b/i;

const containsNegation = (text: string): boolean => NEGATION_REGEX.test(text);

const shouldBlockDangerousReturn = (answer: string): boolean => {
  const mentionsDangerousCargo = DANGEROUS_CARGO_REGEX.test(answer);
  const mentionsReturn = RETURN_REGEX.test(answer);
  const hasNegation = containsNegation(answer);

  return mentionsDangerousCargo && mentionsReturn && !hasNegation;
};

// — Utilitários —————
const createSafeResponse = (): StructuredOutput => ({
  answer: 'Desculpe, não posso processar essa solicitação no momento ou a resposta violou nossas políticas de segurança.',
  source_document: 'N/A',
  confidence_score: 0,
});

const logValidationError = (reason: string, details?: unknown): void => {
  console.error('[response-validator] validation failed:', reason, details ?? 'no details');
};

// — Validação principal —————
export const validateAssistantResponse = (rawLlmResponse: unknown): StructuredOutput => {
  const parseResult = StructuredOutputSchema.safeParse(rawLlmResponse);

  if (!parseResult.success) {
```

```
    logValidationError('structured output validation failed', parseResult.error.flatten());
    return createSafeResponse();
  }

  const parsed = parseResult.data;

  if (shouldBlockDangerousReturn(parsed.answer)) {
    logValidationError('blocked answer: carga perigosa + devolução sem negação explícita', parsed.answer);
    return createSafeResponse();
  }

  return parsed;
};
```